Python + PowerBI – TeleCom Customer Churn Project

The Telecom Customer Churn Analysis project is an end-to-end data analytics project developed using Python and Power BI. The main objective of this project is to analyze customer data, identify patterns related to customer churn, and understand the factors that influence customer retention.

Python was used for data cleaning, preprocessing, and preparation, including handling missing values, correcting data inconsistencies, and preparing a structured dataset for analysis. The cleaned dataset was then imported into Microsoft Power BI to develop an interactive dashboard with KPIs, visualizations, and filters.

Project Details:

Project Title: Telecom Customer Churn Project

Domain: Telecommunications

Dataset Size: 75,000 customer records

Total Columns: 18

Data Analysis Tools: Python and Power BI

Python Environment: Jupyter Notebook

Dashboard: Single-page interactive Power BI dashboard

Project Type: Data Cleaning, Exploratory Data Analysis, Data Visualization, Business Intelligence (BI), Descriptive Analytics

Dataset Preparation Using Python

Column No – 1 (CustomerID)

1) Importing the data and checking datatype

```
[1]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[2]: df["CustomerID"].dtype

[3]: dtype('O')
```

2) Checking missing and duplicate customers

```
[4]: df["CustomerID"].isnull().sum()

[5]: np.int64(0)

[6]: df["CustomerID"].duplicated().sum()

[7]: np.int64(1538)
```

3) Total value count

```
[16]: df["CustomerID"].value_counts().value_counts()

[16]: count
      1    71948
      2     1490
      3       24
      Name: count, dtype: int64
```

4) Checking unique customers and customers who's ids are repeated

```
[18]: df["CustomerID"].nunique()

[18]: 73462

[34]: df["CustomerID"].value_counts()[df["CustomerID"].value_counts() > 1]

[34]: CustomerID
      NET-2105-AX    3
      US-1848-DJ     3
      MDB-9144-G3    3
      NET-9466-A3    3
      MDB-9739-E3    3
                    ..
      MDB-7798-C1    2
      MDB-1589-AY    2
      NET-7036-CZ    2
      TEL-7615-B2    2
      MDB-5726-GZ    2
      Name: count, Length: 1514, dtype: int64
```

Here, the ids of some of the customers are repeated.

5) Inspect repeated IDs – checking are they really duplicates or also contains other information that is different then the same one

```
[13]: df[df["CustomerID"].duplicated(keep=False)].sort_values("CustomerID").head(10)
```

[13]:

	CustomerID	Gender	Age	City	TenureMonths	ContractType	MonthlyCharges	TotalCharges	InternetService	PhoneService	OnlineSecurity
10738	MOB-1028-G3	Male	60	Phoenix	1	Month-to-month	72.71	73.06	DSL	Yes	No
68646	MOB-1028-G3	Male	34	Los Angeles	42	Month-to-month	67.60	2838.72	Fiber optic	Yes	No
25027	MOB-1033-A3	Female	41	Philadelphia	10	Month-to-month	82.40	827.52	Fiber optic	Yes	No
54217	MOB-1033-A3	Female	43	Chicago	46	One year	95.84	4402.17	Fiber optic	Yes	Yes
10414	MOB-1040-HZ	Male	45	San Jose	64	Two year	88.88	5687.37	Fiber optic	Yes	No
33247	MOB-1040-HZ	Male	58	Chicago	24	Month-to-month	96.96	2322.99	Fiber optic	Yes	No
53080	MOB-1159-H2	Male	74	Los Angeles	60	Month-to-month	76.95	4619.13	Fiber optic	Yes	Yes
61829	MOB-1159-H2	Male	42	San Diego	61	Month-to-month	74.61		Fiber optic	Yes	Yes
17481	MOB-1178-CY	Male	44	Columbus	58	Two year	47.33	2738.01	DSL	Yes	Yes
7533	MOB-1178-CY	Female	60	Columbus	69	One year	65.25	4502.28	5G home	Yes	Yes

```
]: df[df["CustomerID"].duplicated(keep=False)].sort_values("CustomerID").head(10)
```

]:

Charges	TotalCharges	InternetService	PhoneService	OnlineSecurity	TechSupport	StreamingTV	PaymentMethod	SupportCalls	Complaints	SatisfactionScore	Churn
72.71	73.06	DSL	Yes	No	No	Yes	Bank transfer (automatic)	4	0	4	No
67.60	2838.72	Fiber optic	Yes	No	No	No	Electronic check	1	2	3	No
82.40	827.52	Fiber optic	Yes	No	No	Yes	Electronic check	4	1	3	Yes
95.84	4402.17	Fiber optic	Yes	Yes	Yes	No	Electronic check	3	1	4	Yes
88.88	5687.37	Fiber optic	Yes	No	No	Yes	Credit card (automatic)	1	0	4	No
96.96	2322.99	Fiber optic	Yes	No	No	Yes	Credit card (automatic)	1	0	5	No
76.95	4619.13	Fiber optic	Yes	Yes	Yes	No	Bank transfer (automatic)	0	1	3	No
74.61		Fiber optic	Yes	Yes	Yes	Yes	Credit card (automatic)	2	0	5	No
47.33	2738.01	DSL	Yes	Yes	Yes	No	UPI	3	2	4	No
65.25	4502.28	5G home	Yes	Yes	Yes	No	Electronic check	2	0	5	No

Here, output confirms that CustomerID has no missing values, but some IDs are assigned to different customer records.

(For example: MOB-1028-G3 appears twice with different ages, cities, tenure, and charges. These are not exact duplicate rows.)

6) Checking exact duplicate records

```
4]: print("Exact Duplicate Rows:", df.duplicated().sum())

    Exact Duplicate Rows: 0
```

Here, the output is 0 meaning there is no single ID that is exactly the same each duplicate IDs contains different customer information.

So, I am not deleting this rows because the repeated IDs have different customer details and I cannot determine whether the repeated IDs are intentional or data errors.

Column No – 2 (Gender)

1) Importing dataset and checking datatype, missing values, unique values present and value count of gender



```
[1]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[4]: df["Gender"].dtype

[4]: dtype('O')

[5]: df["Gender"].isnull().sum()

[5]: np.int64(0)

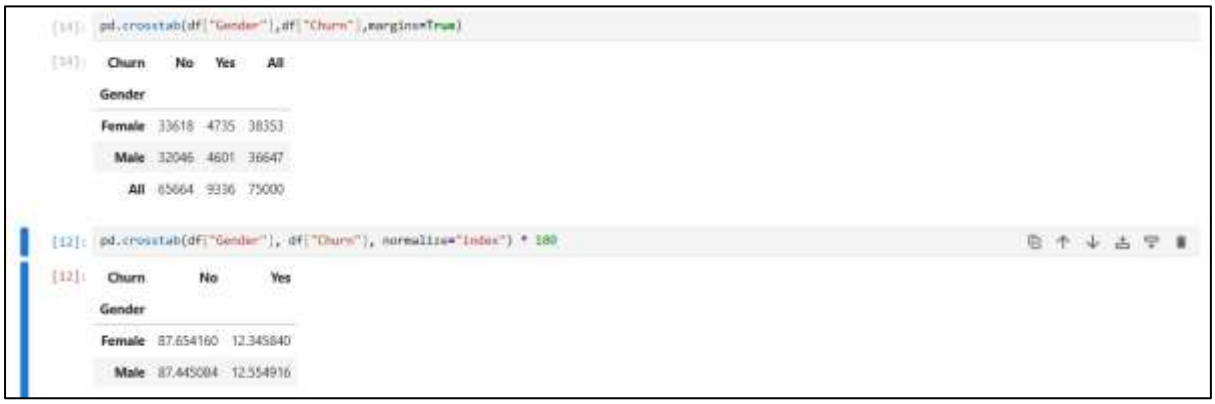
[6]: df["Gender"].unique()

[6]: array(['Female', 'Male'], dtype=object)

[9]: df["Gender"].value_counts()

[9]: Gender
     Female    38353
     Male      36647
     Name: count, dtype: int64
```

2) Churn by gender and churn rate by gender – checking the churn percentage



```
[14]: pd.crosstab(df["Gender"],df["Churn"],margins=True)

[14]:
| Gender | No | Yes | All |
|---|---|---|---|
| Female | 33618 | 4735 | 38353 |
| Male | 32046 | 4601 | 36647 |
| All | 65664 | 9336 | 75000 |

[12]: pd.crosstab(df["Gender"], df["Churn"], normalize="index") * 100

[12]:
| Gender | No | Yes |
|---|---|---|
| Female | 87.654160 | 12.345840 |
| Male | 87.445084 | 12.554916 |
```

Here, male and female customers have very similar churn rates in the dataset.

Column No 3 – (Age)

1) Importing the dataset and checking datatype, null values, minimum and maximum age in the dataset

```
[3]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[5]: df["Age"].dtype

[5]: dtype('int64')

[6]: df["Age"].isnull().sum()

[6]: np.int64(0)

[4]: df["Age"].min()

[4]: 18

[7]: df["Age"].max()

[7]: 120
```

Here, the minimum age 18 is correct but the maximum age is showing 120 that does not look correct.

2) Checking for unusual ages

```
[11]: df[(df["Age"] < 18) | (df["Age"] > 100)]
```

[13]:

	CustomerID	Gender	Age	City	TenureMonths	ContractType	MonthlyCharges	TotalCharges	InternetService	PhoneService	OnlineSecurity	TechSupport
12185	MOB-7004-AY	Male	120	New York	11	Month-to-month	86.41	963.55	Fiber optic	Yes	No	No
18566	TEL-8441-F1	Female	120	Charlotte	65	Month-to-month	59.97	3892.46	DSL	Yes	Yes	Yes
37963	NET-7904-GX	Male	120	Los Angeles	42	One year	58.70	2479.25	DSL	No	No	No
41721	NET-4103-BX	Male	120	Austin	48	One year	84.41	4050.84	Fiber optic	No	No	No
45394	NET-8902-G2	Male	120	San Jose	56	Month-to-month	23.67	1335.75	No	Yes	No internet service	No internet service
46981	TEL-8291-CZ	Female	120	New York	67	Month-to-month	72.08	4841.62	Fiber optic	No	Yes	No
56772	US-4842-G2	Female	120	San Diego	45	Month-to-month	90.52	4080.37	Fiber optic	No	Yes	No
59340	NET-9839-HX	Female	120	San Francisco	72	One year	89.56	6444.44	Fiber optic	Yes	No	No
61284	NET-8533-HZ	Male	120	San Francisco	16	Month-to-month	75.42	1199.73	Fiber optic	Yes	No	No
66252	TEL-6372-E1	Male	120	Austin	58	Month-to-month	34.31	2002.41	No	Yes	No internet service	No internet service

There, are in total 10 records with very unusual ages.

3) Checking the basic statistics and all the values present in the dataset

```
[18]: df["Age"].describe().round(2)

[18]: count    75000.00
      mean        49.90
      std         18.74
      min         18.00
      25%         34.00
      50%         50.00
      75%         66.00
      max        120.00
      Name: Age, dtype: float64

[21]: df["Age"].sort_values().unique()

[21]: array([ 18,  19,  20,  21,  22,  23,  24,  25,  26,  27,  28,  29,  30,
              31,  32,  33,  34,  35,  36,  37,  38,  39,  40,  41,  42,  43,
              44,  45,  46,  47,  48,  49,  50,  51,  52,  53,  54,  55,  56,
              57,  58,  59,  60,  61,  62,  63,  64,  65,  66,  67,  68,  69,
              70,  71,  72,  73,  74,  75,  76,  77,  78,  79,  80,  81,  82,
             120])
```

4) Checking total number of people in each age

```
[40]: df["Age"].value_counts().sort_index()

[40]: Age
      18     1163
      19     1167
      20     1203
      21     1121
      22     1200
             ...
      79     1111
      80     1146
      81     1049
      82     1134
      120      10
      Name: count, Length: 66, dtype: int64
```

We can also use “pd.set_option("display.max_rows", None)
df["Age"].value_counts().sort_index()” to see all the rows.

5) Comparing the average age of customers who churned and didn't churn, and creating age groups that tells us the churn percentage for each age group.

```
[31]: df.groupby("Churn")["Age"].mean().round(2)

[31]: Churn
      No     49.88
      Yes    50.04
      Name: Age, dtype: float64

[36]: df["AgeGroup"] = pd.cut(
          df["Age"],
          bins=[0, 25, 35, 45, 55, 65, 100],
          labels=["18-25", "26-35", "36-45", "46-55", "56-65", "66+"]
      )
      (pd.crosstab(df["AgeGroup"], df["Churn"], normalize="index") * 100).round(2)
```

[36]:

Churn	No	Yes
AgeGroup		
18-25	87.62	12.38
26-35	87.47	12.53
36-45	87.92	12.08
46-55	87.22	12.78
56-65	87.80	12.20
66+	87.40	12.60

6) Replacing the 10 ages of the people who’s age is 120 with the median age which is 50 because it seems to be something valid and checking the maximum age

```
[43]: import numpy as np
      df.loc[df["Age"] == 120, "Age"] = np.nan

[44]: df["Age"].isnull().sum()

[44]: np.int64(10)

[45]: df["Age"] = df["Age"].fillna(50)

[46]: df["Age"].isnull().sum()

[46]: np.int64(0)

[47]: df["Age"].max()

[47]: 82.0
```

Now, the maximum age 82 seems to be correct and the age 120 is also changed to 50.

Column No 4 – (City)

1) Importing dataset and checking datatype, null values, unique values and value count of different cities

```
[1]: import numpy as np
     import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[2]: df["City"].dtype

[2]: dtype('O')

[4]: df["City"].isnull().sum()

[4]: np.int64(0)

[5]: df["City"].unique()

[5]: array(['San Jose', 'Austin', 'San Antonio', 'Los Angeles', 'Chicago',
            'San Francisco', 'Houston', 'Philadelphia', 'New York', 'Columbus',
            'Charlotte', 'San Diego', 'Dallas', 'Phoenix', 'Jacksonville'],
           dtype=object)

[6]: df["City"].value_counts()

[6]: City
     Charlotte       5140
     Chicago         5121
     San Jose        5067
     Philadelphia    5066
     San Francisco   5032
     Los Angeles     4997
     San Antonio     4994
     Houston         4992
     San Diego       4980
     Jacksonville    4964
     New York        4958
     Columbus        4956
     Phoenix         4930
     Dallas          4917
     Austin          4896
     Name: count, dtype: int64

[7]: (df["City"] == "New York").sum()

[7]: np.int64(4958)

[8]: df["City"].nunique()

[8]: 15
```

2) Checking customer churn by city and calculating churn percentage by city

```
pd.crosstab(df["City"], df["Churn"])
```

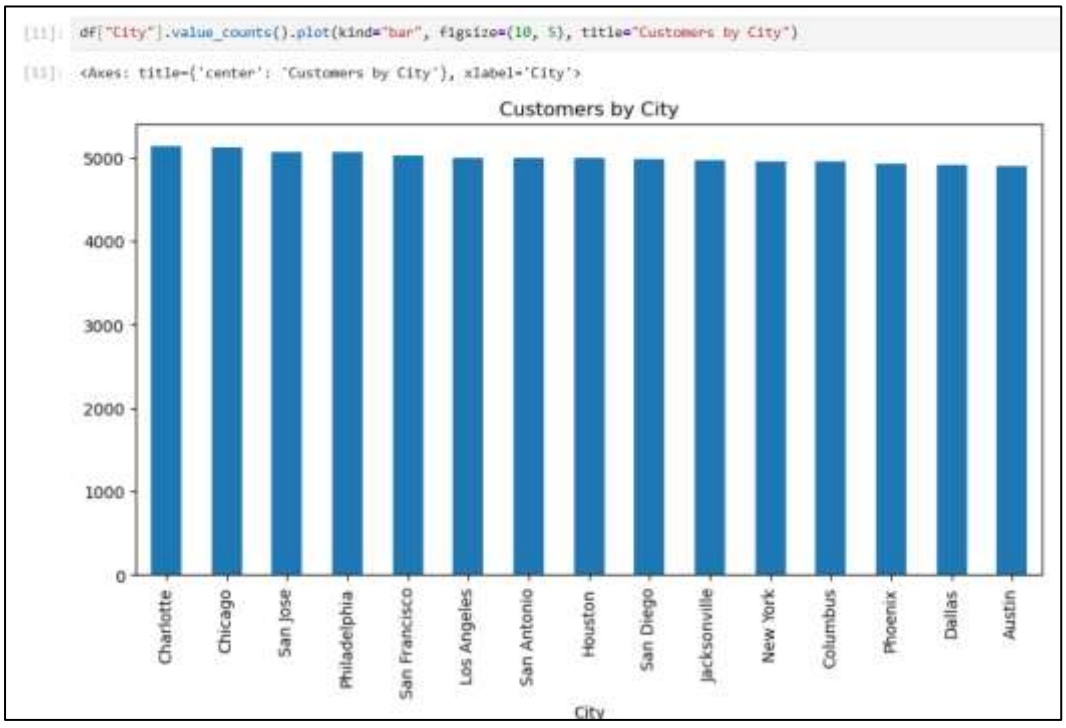
Churn	No	Yes
City		
Austin	4272	624
Charlotte	4539	601
Chicago	4491	630
Columbus	4361	595
Dallas	4294	623
Houston	4346	646
Jacksonville	4369	595
Los Angeles	4339	658
New York	4325	633
Philadelphia	4459	607
Phoenix	4266	664
San Antonio	4364	630
San Diego	4403	577
San Francisco	4401	621
San Jose	4435	632

```
pd.crosstab(df["City"], df["Churn"], normalize="index") * 100
```

Churn	No	Yes
City		
Austin	87.254902	12.745098
Charlotte	88.307393	11.692607
Chicago	87.697715	12.302285
Columbus	87.994350	12.005650
Dallas	87.329673	12.670327
Houston	87.059295	12.940705
Jacksonville	88.013699	11.986301
Los Angeles	86.832099	13.167901
New York	87.232755	12.767245
Philadelphia	88.018160	11.981840
Phoenix	86.531440	13.468560
San Antonio	87.384862	12.615138
San Diego	88.413655	11.586345
San Francisco	87.634409	12.365591
San Jose	87.527136	12.472864

3) Visualizing customers by city

```
[11]: df["City"].value_counts().plot(kind="bar", figsize=(10, 5), title="Customers by City")
[11]: <Axes: title={'center': 'Customers by City'}, xlabel='City'>
```



Column No 5 – (TenureMonths)

1) Importing dataset and checking datatype, missing values, minimum and maximum values, basic statistics, total negative values, zero-tenure customers

```
[1]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[3]: df["TenureMonths"].dtype

[3]: dtype('int64')

[4]: df["TenureMonths"].isnull().sum()

[4]: np.int64(0)

[5]: df["TenureMonths"].min(), df["TenureMonths"].max()

[5]: (-1, 72)

[6]: df["TenureMonths"].describe()

[6]: count    75000.000000
     mean        35.825093
     std         21.039020
     min         -1.000000
     25%         18.000000
     50%         36.000000
     75%         54.000000
     max         72.000000
     Name: TenureMonths, dtype: float64

[7]: (df["TenureMonths"] < 0).sum()

[7]: np.int64(15)

[9]: (df["TenureMonths"] == 0).sum()

[9]: np.int64(1003)
```

Here, there are 15 values that are less then 0 which is -1 and tenure months can not be in negative and around 1003 values are 0 which is potentially valid because the dataset may includes brand-new customers.

2) Checking churn by tenure and creating tenure group for analysis

```
df.groupby("Churn")["TenureMonths"].mean()

Churn
No     37.589592
Yes    23.416238
Name: TenureMonths, dtype: float64
```

```
df["TenureGroup"] = pd.cut(
    df["TenureMonths"],
    bins=[-1, 6, 12, 24, 48, float("inf")],
    labels=["0-6", "7-12", "13-24", "25-48", "49+"]
)
pd.crosstab(df["TenureGroup"], df["Churn"])
```

Churn	No	Yes
TenureGroup		
0-6	5184	1928
7-12	4893	1430
13-24	10228	2231
25-48	22235	2607
49+	23110	1139

3) Checking churn percentage by tenure group

```
pd.crosstab(df["TenureGroup"], df["Churn"], normalize="index") * 100
```

Churn	No	Yes
TenureGroup		
0-6	72.933913	27.066087
13-24	82.093266	17.906734
25-48	89.505676	10.494324
49+	95.302899	4.697101
7-12	77.384153	22.615847

4) Examine -1 tenure months value and Replacing -1 values with 0 because it can represents new customers and since -1 value seems to be incorrect it might be an input error

```
df.loc[df["TenureMonths"] < 0]
```

	CustomerID	Gender	Age	City	TenureMonths	ContractType	MonthlyCharges	TotalCharges	InternetService	PhoneService	OnlineSecurity	TechSuppo
875	MOB-2712-GX	Male	69	Philadelphia	-1	One year	86.26	3644.74	Fiber optic	Yes	No	Y
11368	MOB-7027-HX	Male	52	San Diego	-1	Two year	83.91	5430.18	Fiber optic	No	Yes	I
26026	TEL-9120-CX	Male	74	Charlotte	-1	Month-to-month	102.94	1858.49	Fiber optic	Yes	Yes	Y
26744	MOB-6303-DY	Female	64	Dallas	-1	Month-to-month	53.35	1615.5	DSL	Yes	Yes	I
29021	TEL-4312-BZ	Male	54	Houston	-1	Two year	57.01	3988.33	DSL	Yes	No	Y
29333	TEL-8358-HJ	Female	70	Charlotte	-1	One year	82.48	4388.37	Fiber optic	Yes	Yes	I
30347	NET-1824-GX	Female	44	San Jose	-1	Month-to-month	60.95	1883.68	5G home	Yes	No	I
33508	MOB-2205-BX	Female	32	Chicago	-1	Month-to-month	44.71	2278.71	DSL	Yes	No	I
43134	MOB-1690-FV	Female	77	Columbus	-1	Two year	91.10	1550.63	Fiber optic	Yes	Yes	I
47266	MOB-2492-GZ	Female	23	Dallas	-1	One year	63.56	451.19	5G home	Yes	No	I
51514	US-1364-DJ	Female	47	San Jose	-1	One year	80.06	2919.8	Fiber optic	Yes	No	I
52632	TEL-9511-FX	Male	35	Los Angeles	-1	Two year	77.44	4889.75	5G home	Yes	No	I
66514	NET-9307-EZ	Male	21	Philadelphia	-1	Month-to-month	51.30	0.0	DSL	Yes	No	I
73455	US-3346-C2	Female	63	San Francisco	-1	Month-to-month	65.74	3373.76	5G home	Yes	No	I
73826	NET-4213-DX	Female	18	Houston	-1	Month-to-month	43.07	878.01	DSL	Yes	No	I

Only shows CustomerID and TenureMonths columns

df.loc[df["TenureMonths"] < 0, ["CustomerID", "TenureMonths"]]

```
df.loc[df["TenureMonths"] < 0, "TenureMonths"] = 0

(df["TenureMonths"] < 0).sum()

np.int64(0)
```

Column No 6 – (ContractType)

1) Importing dataset and checking missing values, datatype, unique values and value count of customers by contract type

```
[1]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[3]: df["ContractType"].dtype

[3]: dtype('O')

[6]: df["ContractType"].isnull().sum()

[6]: np.int64(0)

[9]: df["ContractType"].unique()

[9]: array(['Month-to-month', 'One year', 'Two year'], dtype=object)

[10]: df["ContractType"].value_counts()

[10]: ContractType
      Month-to-month    41089
      One year          19004
      Two year          14907
      Name: count, dtype: int64
```

2) Checking churn counts by contract type and churn rate (percentage)

```
[11]: pd.crosstab(df["ContractType"], df["Churn"])
```

[11]:

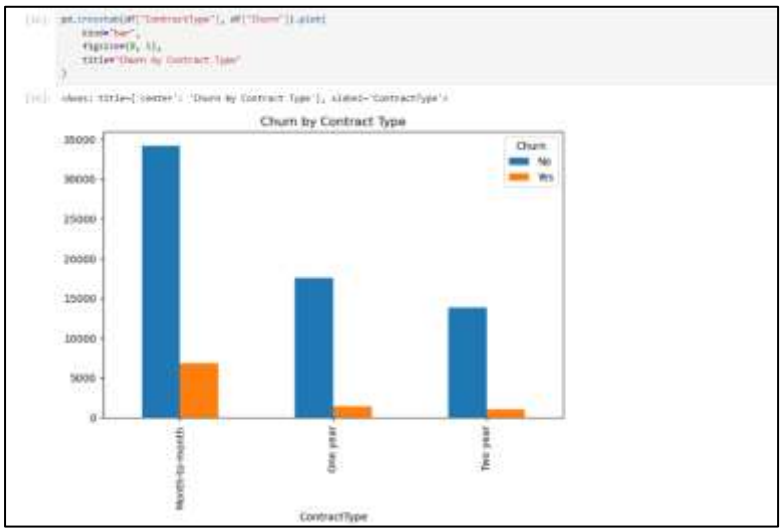
ContractType	Churn	No	Yes
Month-to-month		34231	6858
One year		17594	1410
Two year		13839	1068

```
[15]: pd.crosstab(df["ContractType"], df["Churn"], normalize="index") * 100
```

[15]:

ContractType	Churn	No	Yes
Month-to-month		83.309402	16.690598
One year		92.580509	7.419491
Two year		92.835581	7.164419

3) Visualizing churn by contract type



Column No 7 – (MonthlyCharges)

1) Importing dataset and checking datatype, missing values, minimum and maximum values, basic statistics and checking that less then 0 values exists or not and comparing average monthly charges by churn

```
[1]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[3]: df["MonthlyCharges"].dtype

[3]: dtype('float64')

[5]: df["MonthlyCharges"].isnull().sum()

[5]: np.int64(0)

[6]: df["MonthlyCharges"].min(), df["MonthlyCharges"].max()

[6]: (18.25, 118.75)

[8]: df["MonthlyCharges"].describe().round(2)

[8]: count    75000.00
     mean        63.39
     std         23.04
     min         18.25
     25%         49.25
     50%         64.69
     75%         80.93
     max        118.75
     Name: MonthlyCharges, dtype: float64

[9]: (df["MonthlyCharges"] < 0).sum()

[9]: np.int64(0)

[10]: df.groupby("Churn")["MonthlyCharges"].mean()

[10]: Churn
      No     63.016716
      Yes    65.983636
      Name: MonthlyCharges, dtype: float64
```

2) Grouping monthly charges and Calculating churn percentage by charge group

```
[22]: df["ChargeGroup"] = pd.cut(
          df["MonthlyCharges"],
          bins=[-1, 30, 60, 90, 120, float("inf")],
          labels=["0-30", "31-60", "61-90", "91-120", "120+"]
      )
      pd.crosstab(df["ChargeGroup"], df["Churn"], normalize="index") * 100
```

[22]:

Churn	No	Yes
ChargeGroup		
0-30	89.207353	10.792647
31-60	88.898357	11.101643
61-90	86.839563	13.160437
91-120	85.379325	14.620675

Column No 8 – (TotalCharges)

1) Importing dataset and checking datatype, missing values, blank-space values, unique values (which displays the first 20 unique values) and basic statistics

```
import pandas as pd
df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")
```

```
df["TotalCharges"].dtype
```

```
dtype('O')
```

```
df["TotalCharges"].isnull().sum()
```

```
np.int64(897)
```

```
(df["TotalCharges"].astype(str).str.strip() == "").sum()
```

```
np.int64(375)
```

```
df["TotalCharges"].unique()[:20]
```

```
array(['2055.59', '1564.91', '179.22', '3170.32', '3540.84', '1363.61',
       '4530.59', '2613.19', '5196.21', '3402.92', '1063.86', '4552.45',
       '98.23', '1298.73', '4969.68', '2921.33', '910.02', '2496.2',
       '2069.07', '3020.97'], dtype=object)
```

```
df["TotalCharges"].describe()
```

```
count     74103
unique    67873
top         0.0
freq        985
Name: TotalCharges, dtype: object
```

Here, the datatype is object which is incorrect because total charges contains numeric/decimal values and also there are some 897 null values and 375 blank values.

2) Converting the column to numeric

```
[18]: df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

[19]: df["TotalCharges"].dtype

[19]: dtype('float64')

[20]: df["TotalCharges"].isnull().sum()

[20]: np.int64(1272)
```

Here, blank spaces become missing values.

3) Investigating the missing values

```
df.loc[df["TotalCharges"].isnull(), "TenureMonths"].value_counts().sort_index()

TenureMonths
0     19
1     21
2     20
3     25
4     19
      ..
68    11
69    17
70    24
71    12
72    17
Name: count, Length: 73, dtype: int64
```

Here, output shows that 1,272 missing total charges values occur across tenure months values from 0 to 72 months.

4) Checking missing charges for customers with zero tenure

```
[23]: df.loc[df["TotalCharges"].isnull() & (df["TenureMonths"] == 0)]
```

[27]:

	CustomerID	Gender	Age	City	TenureMonths	ContractType	MonthlyCharges	TotalCharges	InternetService	PhoneService	OnlineSecurity	TechSupport
8578	MC8-1367-D1	Male	69	Dallas	0	One year	26.33	NaN	No	Yes	No internet service	No internet service
11380	US-3311-DY	Male	18	San Antonio	0	One year	43.41	NaN	DSL	Yes	No	No
14818	TEL-6534-BY	Male	27	Los Angeles	0	Month-to-month	79.97	NaN	Fiber optic	Yes	Yes	Yes
16128	TEL-9093-AY	Female	80	San Francisco	0	Two year	92.75	NaN	Fiber optic	Yes	Yes	No
16168	NET-4451-EX	Male	80	New York	0	Month-to-month	33.18	NaN	No	No	No internet service	No internet service
26626	NET-7445-E1	Female	52	Austin	0	One year	96.77	NaN	Fiber optic	Yes	No	No
31071	NET-6177-E1	Male	28	San Antonio	0	Two year	102.00	NaN	Fiber optic	Yes	No	Yes
33275	NET-8868-F3	Female	43	San Jose	0	Month-to-month	44.71	NaN	DSL	No	No	No
33445	US-4866-GY	Male	68	Dallas	0	Month-to-month	54.40	NaN	DSL	Yes	No	No
33614	TEL-6577-G1	Male	61	Houston	0	One year	90.52	NaN	Fiber optic	Yes	No	Yes
33648	NET-3880-E3	Male	19	Dallas	0	Month-to-month	57.28	NaN	DSL	No	No	Yes
34634	NET-7163-FX	Male	47	Charlotte	0	One year	82.04	NaN	Fiber optic	Yes	No	Yes

Here, around 19 customers are with missing total charges and having 0 tenure months

So, either these customers may be new who have not yet generated a total charge.

5) Checking the other 1,253 missing values with positive tenure months but have missing total charges

```
[4]: df.loc[
         df["TotalCharges"].isnull() & (df["TenureMonths"] > 0),
         ["TenureMonths", "MonthlyCharges", "TotalCharges"]
     ].head(10)
```

[4]:

	TenureMonths	MonthlyCharges	TotalCharges
36	51	102.58	NaN
47	14	44.46	NaN
134	49	62.40	NaN
142	39	59.01	NaN
174	39	57.04	NaN
213	42	84.83	NaN
250	29	58.63	NaN
274	22	69.89	NaN
349	28	98.32	NaN
377	70	29.21	NaN

Here, I can fill the total charges values with the median who's tenure month is > 0

through:

median_value = df["TotalCharges"].median()

df.loc[(df["TotalCharges"].isnull()) & (df["TenureMonths"] > 0), "TotalCharges"]=median_value

and fill 0 for the customers who's tenure month and total charges are 0 through:

df.loc[(df["TotalCharges"].isnull()) & (df["TenureMonths"] == 0), "TotalCharges"] = 0

So, for now I will replace all the null values with the median value which is 1960.94 because it is difficult to determine if they are new customers or old ones.

6) Filling the missing values with the median and finally checking the current stats.

```
[51]: median_value = df["TotalCharges"].median()

      df["TotalCharges"] = df["TotalCharges"].fillna(median_value)
```

```
[45]: print("Missing Values:", df["TotalCharges"].isnull().sum())
      print("Data Type:", df["TotalCharges"].dtype)
      print("Minimum:", df["TotalCharges"].min())
      print("Maximum:", df["TotalCharges"].max())
      print("Median:", df["TotalCharges"].median())
```

```
Missing Values: 0
Data Type: float64
Minimum: 0.0
Maximum: 8550.32
Median: 1960.9450000000002
```

Column No 9 – (InternetService)

1) Importing dataset and checking datatype, missing values, unique values and value count by internet services

```
[1]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[7]: df["InternetService"].dtype

[7]: dtype('O')

[8]: df["InternetService"].isnull().sum()

[8]: np.int64(0)

[9]: df["InternetService"].unique()

[9]: array(['DSL', 'Fiber optic', '5G home', 'No'], dtype=object)

[10]: df["InternetService"].value_counts()

[10]: InternetService
      Fiber optic    28355
      DSL            21223
      5G home        13436
      No             11986
      Name: count, dtype: int64
```

2) Checking churn counts by internet service and calculating churn percentages

```
pd.crosstab(df["InternetService"], df["Churn"])
```

Churn	No	Yes
InternetService		
5G home	11976	1460
DSL	18926	2297
Fiber optic	24091	4264
No	10671	1315

```
pd.crosstab(df["InternetService"], df["Churn"], normalize="index") * 100
```

Churn	No	Yes
InternetService		
5G home	89.133671	10.866329
DSL	89.176836	10.823164
Fiber optic	84.962088	15.037912
No	89.028867	10.971133

Column No 10 – (PhoneService)

1) Importing dataset and checking datatype, missing values, unique values, and value count of phone service

```
[1]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[3]: df["PhoneService"].dtype

[3]: dtype('O')

[4]: df["PhoneService"].isnull().sum()

[4]: np.int64(0)

[5]: df["PhoneService"].unique()

[5]: array(['Yes', 'No'], dtype=object)

[6]: df["PhoneService"].value_counts()

[6]: PhoneService
     Yes    67469
     No      7531
     Name: count, dtype: int64
```

2) Checking churn count and calculating churn percentages

```
[7]: pd.crosstab(df["PhoneService"], df["Churn"])
```

[7]:

PhoneService	Churn	No	Yes
No		6549	982
Yes		59115	8354

```
[8]: pd.crosstab(df["PhoneService"], df["Churn"], normalize="index") * 100
```

[8]:

PhoneService	Churn	No	Yes
No		86.960563	13.039437
Yes		87.618017	12.381983

Column No 11 – (OnlineSecurity)

1) Importing dataset, checking datatype, missing values, unique values, value counts of online security

```
[1]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[3]: df["OnlineSecurity"].dtype

[3]: dtype('O')

[4]: df["OnlineSecurity"].isnull().sum()

[4]: np.int64(450)

[5]: df["OnlineSecurity"].unique()

[5]: array(['Yes', 'No', 'No internet service', nan], dtype=object)

[6]: df["OnlineSecurity"].value_counts()

[6]: OnlineSecurity
     No                  37639
     Yes                 24981
     No internet service 11930
     Name: count, dtype: int64
```

2) Checking online security against churn and calculating churn percentage

```
[7]: pd.crosstab(df["OnlineSecurity"], df["Churn"])
```

[7]:

OnlineSecurity	Churn No	Churn Yes
No	32778	4861
No internet service	10622	1308
Yes	21880	3101

```
[8]: pd.crosstab(df["OnlineSecurity"], df["Churn"], normalize="index") * 100
```

[8]:

OnlineSecurity	Churn No	Churn Yes
No	87.085204	12.914796
No internet service	89.036044	10.963956
Yes	87.586566	12.413434

Here, there are 450 missing values.

3) Checking whether those customers have internet service

```
[10]: df.loc[df["OnlineSecurity"].isnull(), "InternetService"].value_counts()

[10]: InternetService
      Fiber optic    173
      DSL            140
      5G home         81
      No              56
      Name: count, dtype: int64
```

Here, 56 customers don’t use or have any internet service So, I cannot assume the missing values because it might be a data-entry issue.

4) Checking the other services columns

```
df.loc[df["OnlineSecurity"].isnull(),
       ["InternetService", "PhoneService", "TechSupport", "StreamingTV"]].head(10)
```

	InternetService	PhoneService	TechSupport	StreamingTV
508	DSL	Yes	Yes	No
731	Fiber optic	Yes	No	No
781	5G home	Yes	No	No
814	No	No	No internet service	No internet service
950	Fiber optic	No	No	No
1081	5G home	Yes	Yes	No
1251	Fiber optic	Yes	No	Yes
1496	Fiber optic	Yes	No	No
1521	Fiber optic	Yes	No	No
1536	Fiber optic	Yes	No	Yes

Here, output shows that the missing Online Security values occurs alongside different combinations of other service values. So, i cannot guess replacements based on these rows alone.

5) Filling missing values with "Unknown" because, I cannot reliably determine whether those 450 customers have online security or not.

```
[13]: df["OnlineSecurity"] = df["OnlineSecurity"].fillna("Unknown")

[14]: print("Missing Values:", df["OnlineSecurity"].isnull().sum())
      print("\nValue Counts:")
      print(df["OnlineSecurity"].value_counts())
```

```
Missing Values: 0

Value Counts:
OnlineSecurity
No                     37639
Yes                    24981
No internet service    11930
Unknown                  450
Name: count, dtype: int64
```

Column No 12 – (TechSupport)

1) Importing dataset and checking datatype, missing values, unique values, and each value count

```
[1]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[3]: df["TechSupport"].dtype

[3]: dtype('O')

[4]: df["TechSupport"].isnull().sum()

[4]: np.int64(600)

[5]: df["TechSupport"].unique()

[5]: array(['No', 'Yes', 'No internet service', nan], dtype=object)

[6]: df["TechSupport"].value_counts()

[6]: TechSupport
     No                  37278
     Yes                 25227
     No internet service    11895
     Name: count, dtype: int64
```

2) Checking tech support against churn and calculating churn percentage

```
[7]: pd.crosstab(df["TechSupport"], df["Churn"])

[7]:
```

TechSupport	Churn	No	Yes
No		32549	4729
No internet service		10590	1305
Yes		22004	3223

```
[8]: pd.crosstab(df["TechSupport"], df["Churn"], normalize="index") * 100

[8]:
```

TechSupport	Churn	No	Yes
No		87.314234	12.685766
No internet service		89.029004	10.970996
Yes		87.224006	12.775994

Here, 600 are the original actual missing values.

3) Filling missing values with "Unknown" because, I cannot reliably determine whether those 600 customers have tech support or not.

```
[9]: df["TechSupport"] = df["TechSupport"].fillna("Unknown")

[10]: print("Missing Values:", df["TechSupport"].isnull().sum())
      print("\nValue Counts:")
      print(df["TechSupport"].value_counts())

      Missing Values: 0

      Value Counts:
      TechSupport
      No                     37278
      Yes                    25227
      No internet service    11895
      Unknown                  600
      Name: count, dtype: int64
```

Column No 13 – (StreamingTV)

1) Importing dataset and checking datatype, missing values, unique values and value count of streaming tv

```
[1]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[3]: df["StreamingTV"].dtype

[3]: dtype('O')

[4]: df["StreamingTV"].isnull().sum()

[4]: np.int64(0)

[5]: df["StreamingTV"].unique()

[5]: array(['Yes', 'No', 'No internet service'], dtype=object)

[6]: df["StreamingTV"].value_counts()

[6]: StreamingTV
     No                     37681
     Yes                    25333
     No internet service    11986
     Name: count, dtype: int64
```

2) Checking Streaming TV against churn and Calculating churn percentage

```
[7]: pd.crosstab(df["StreamingTV"], df["Churn"])
```

[7]:

StreamingTV \ Churn	No	Yes
No	32882	4799
No internet service	10671	1315
Yes	22111	3222

```
[8]: pd.crosstab(df["StreamingTV"], df["Churn"], normalize="index") * 100
```

[8]:

StreamingTV \ Churn	No	Yes
No	87.264138	12.735862
No internet service	89.028867	10.971133
Yes	87.281412	12.718588

Column No 14 – (PaymentMethod)

1) Importing dataset and checking datatype, missing values, unique values and value count of payment method

```
[1]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[3]: df["PaymentMethod"].dtype

[3]: dtype('O')

[4]: df["PaymentMethod"].isnull().sum()

[4]: np.int64(300)

[5]: df["PaymentMethod"].unique()

[5]: array(['Electronic check', nan, 'Credit card (automatic)',
            'Bank transfer (automatic)', 'UPI'], dtype=object)

[6]: df["PaymentMethod"].value_counts()

[6]: PaymentMethod
     Electronic check             25255
     UPI                          16645
     Bank transfer (automatic)    16487
     Credit card (automatic)      16313
     Name: count, dtype: int64
```

2) Checking payment method against churn and calculating churn percentage

```
[7]: pd.crosstab(df["PaymentMethod"], df["Churn"])
```

PaymentMethod	Churn	No	Yes
Bank transfer (automatic)		14633	1854
Credit card (automatic)		14451	1862
Electronic check		21528	3727
UPI		14787	1858

```
[8]: pd.crosstab(df["PaymentMethod"], df["Churn"], normalize="index") * 100
```

PaymentMethod	Churn	No	Yes
Bank transfer (automatic)		88.754776	11.245224
Credit card (automatic)		88.585790	11.414210
Electronic check		85.242526	14.757474
UPI		88.837489	11.162511

Here, there are in total 300 missing values.

3) Inspecting null payment method records

```
[9]: df.loc[df["PaymentMethod"].isnull(),
           ["ContractType", "MonthlyCharges", "InternetService", "Churn"]].head(10)
```

[8]:

	ContractType	MonthlyCharges	InternetService	Churn
1	Month-to-month	82.83	Fiber optic	No
630	Month-to-month	23.80	No	No
887	One year	65.27	DSL	No
1179	Month-to-month	32.78	No	No
1214	One year	50.83	DSL	No
1392	Two year	66.05	DSL	No
1533	Month-to-month	23.77	No	No
1545	One year	44.47	DSL	No
1640	One year	69.80	DSL	No
1903	Month-to-month	62.25	DSL	No

These outputs show that the customers with missing payment method values have different contract types, monthly charges, and internet services. So, I cannot determine which payment methods this customers used.

4) Filling the 300 missing values with “Unknown” and checking the value counts of each value

```
]: df["PaymentMethod"] = df["PaymentMethod"].fillna("Unknown")

]: df["PaymentMethod"].isnull().sum()

]: np.int64(0)

]: df["PaymentMethod"].value_counts()

]: PaymentMethod
   Electronic check             25255
   UPI                          16645
   Bank transfer (automatic)    16487
   Credit card (automatic)      16313
   Unknown                        300
   Name: count, dtype: int64
```

Column No 15 – (SupportCalls)

1) Importing dataset and checking datatype, missing values, unique values, basic statistics and unusual values (<0)

```
import pandas as pd
df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")
```

```
df["SupportCalls"].dtype
```

```
dtype('int64')
```

```
df["SupportCalls"].isnull().sum()
```

```
np.int64(0)
```

```
df["SupportCalls"].unique()
```

```
array([ 1,  0,  4,  3,  2,  6,  7,  5, 10,  8,  9])
```

```
df["SupportCalls"].describe().round(2)
```

```
count    75000.00
mean         1.80
std          1.34
min          0.00
25%          1.00
50%          2.00
75%          3.00
max         10.00
Name: SupportCalls, dtype: float64
```

```
(df["SupportCalls"] < 0).sum()
```

```
np.int64(0)
```

2) Checking support calls against churn and calculating churn percentage

```
]: pd.crosstab(df["SupportCalls"], df["Churn"])
```

]:

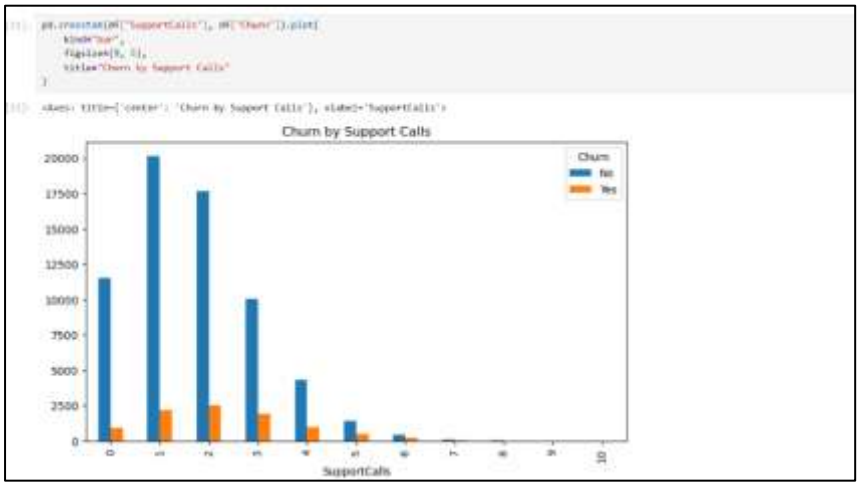
SupportCalls	No	Yes
0	11513	958
1	20122	2194
2	17656	2535
3	10039	1923
4	4333	992
5	1436	467
6	436	202
7	102	47
8	26	11
9	1	4
10	0	3

```
]: pd.crosstab(df["SupportCalls"], df["Churn"], normalize="index") * 100
```

]:

SupportCalls	No	Yes
0	92.318178	7.681822
1	90.168489	9.831511
2	87.444901	12.555099
3	83.924093	16.075907
4	81.370892	18.629108
5	75.459800	24.540200
6	68.338558	31.661442
7	68.456376	31.543624
8	70.270270	29.729730
9	20.000000	80.000000
10	0.000000	100.000000

3) Creating a chart – churn by support calls



Column No 16 – (Complaints)

1) Importing dataset and checking datatype, missing values, unique values, and value counts of complaints

```
[1]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[3]: df["Complaints"].dtype

[3]: dtype('int64')

[4]: df["Complaints"].isnull().sum()

[4]: np.int64(0)

[5]: df["Complaints"].unique()

[5]: array([0, 1, 3, 2, 4, 5])

[6]: df["Complaints"].value_counts()

[6]: Complaints
     0    41275
     1    24670
     2     7332
     3     1480
     4      217
     5       26
     Name: count, dtype: int64
```

2) Checking basic statistics and unusual values (<0)

```
[7]: df["Complaints"].describe().round(2)

[7]: count    75000.00
     mean         0.60
     std          0.77
     min          0.00
     25%          0.00
     50%          0.00
     75%          1.00
     max          5.00
     Name: Complaints, dtype: float64

[8]: (df["Complaints"] < 0).sum()

[8]: np.int64(0)
```

3) Checking complaints against churn and calculating churn percentage

```
[9]: pd.crosstab(df["Complaints"], df["Churn"])
```

[9]:

Complaints	No	Yes
0	38173	3102
1	21110	3560
2	5408	1924
3	887	593
4	81	136
5	5	21

```
[10]: pd.crosstab(df["Complaints"], df["Churn"], normalize="index") * 100
```

[10]:

Complaints	No	Yes
0	92.484555	7.515445
1	85.569518	14.430482
2	73.758865	26.241135
3	59.932432	40.067568
4	37.327189	62.672811
5	19.230769	80.769231

Column No 17 – (SatisfactionScore)

1) Importing dataset and checking datatype, missing values, unique values, and value counts of each value

```
[1]: import pandas as pd
     df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[3]: df["SatisfactionScore"].dtype

[3]: dtype('int64')

[4]: df["SatisfactionScore"].isnull().sum()

[4]: np.int64(0)

[5]: df["SatisfactionScore"].unique()

[5]: array([5, 4, 3, 2, 1])

[8]: df["SatisfactionScore"].value_counts().sort_index()

[8]: SatisfactionScore
     1      472
     2     3253
     3    13209
     4    27838
     5    30228
     Name: count, dtype: int64
```

2) Checking basic statistics and invalid values

```
[10]: df["SatisfactionScore"].describe().round(3)

[10]: count    75000.000
      mean         4.121
      std          0.891
      min          1.000
      25%          4.000
      50%          4.000
      75%          5.000
      max          5.000
      Name: SatisfactionScore, dtype: float64

[11]: ((df["SatisfactionScore"] < 1) | (df["SatisfactionScore"] > 5)).sum()

[11]: np.int64(0)
```

3) Checking satisfaction against churn and calculating churn percentage

```
[12]: pd.crosstab(df["SatisfactionScore"], df["Churn"])

[12]:
```

SatisfactionScore	No	Yes
1	242	230
2	2111	1142
3	10463	2746
4	24560	3278
5	28288	1940

```
[13]: pd.crosstab(df["SatisfactionScore"], df["Churn"], normalize="index") * 100

[13]:
```

SatisfactionScore	No	Yes
1	51.271186	48.728814
2	64.893944	35.106056
3	79.211144	20.788856
4	88.224729	11.775271
5	93.582109	6.417891

Column No 18 – (Churn)

1) Importing dataset and checking datatype, missing values, unique values, value count of each value and calculating churn percentages and checking for invalid values

```
[11]: import matplotlib.pyplot as plt
      import pandas as pd
      df = pd.read_csv("D:\\EXCEL PROJECTS\\Fifth New Project\\telecom_dataset.csv")

[3]: df["Churn"].dtype

[3]: dtype('O')

[4]: df["Churn"].isnull().sum()

[4]: np.int64(0)

[5]: df["Churn"].unique()

[5]: array(['No', 'Yes'], dtype=object)

[6]: df["Churn"].value_counts()

[6]: Churn
     No     65664
     Yes     9336
     Name: count, dtype: int64

[7]: df["Churn"].value_counts(normalize=True) * 100

[7]: Churn
     No     87.552
     Yes    12.448
     Name: proportion, dtype: float64

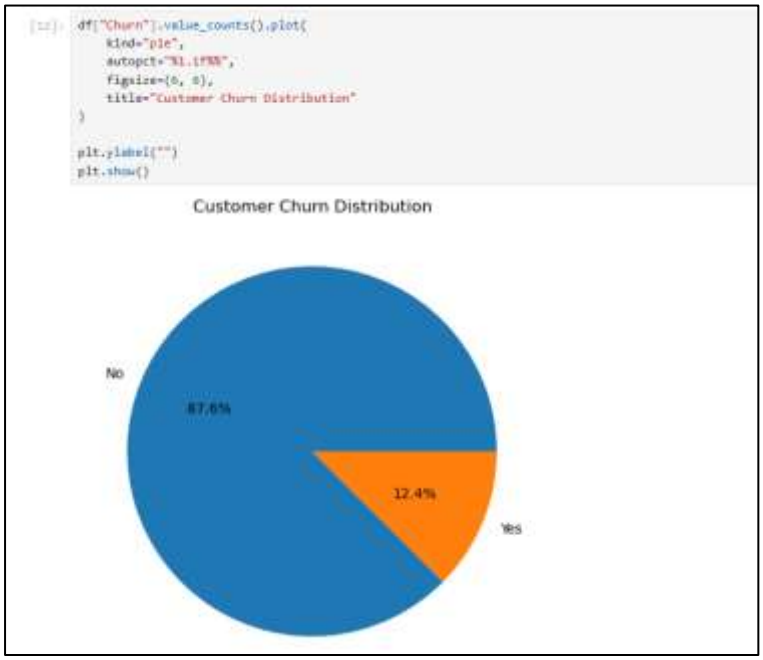
[8]: (~df["Churn"].isin(["Yes", "No"])).sum()

[8]: np.int64(0)
```

2) Creating a pie chart - customer churn distribution

```
[12]: df["Churn"].value_counts().plot(
          kind="pie",
          autopct="%1.1f%%",
          figsize=(6, 6),
          title="Customer Churn Distribution"
      )

      plt.ylabel("")
      plt.show()
```



Exporting The Cleaned File In The Local System:

1) Checking whether the current data frame is actually cleaned or not

```
74]: print("Dataset Size:", df.shape)

     print("\nMissing Values:")
     print(df.isnull().sum())

     print("\nAge 120 values:", (df['Age'] == 120).sum())
     print("Negative Tenure:", (df['TenureMonths'] < 0).sum())
     print("OnlineSecurity missing:", df['OnlineSecurity'].isnull().sum())
     print("TechSupport missing:", df['TechSupport'].isnull().sum())
     print("PaymentMethod missing:", df['PaymentMethod'].isnull().sum())
```

```
Dataset Size: (75000, 20)

Missing Values:
CustomerID          0
Gender              0
Age                 0
City                0
TenureMonths        0
ContractType        0
MonthlyCharges      0
TotalCharges        0
InternetService     0
PhoneService        0
OnlineSecurity      0
TechSupport         0
StreamingTV         0
PaymentMethod       0
SupportCalls        0
Complaints          0
SatisfactionScore   0
Churn               0
AgeGroup            0
ChargeGroup         0
dtype: int64

Age 120 values: 0
Negative Tenure: 0
OnlineSecurity missing: 0
TechSupport missing: 0
PaymentMethod missing: 0
```

2) Hence, the agegroup and chargegroup columns where already created during the dataset cleaning stage but, I am creating them again properly.

```
[158]: df['AgeGroup'] = pd.cut(
           df['Age'],
           bins=[17, 30, 45, 60, 75, 120],
           labels=['18-30', '31-45', '46-60', '61-75', '76+']
       )

[159]: df['AgeGroup'].isnull().sum()

[159]: np.int64(0)
```

```
[167]: df['ChargeGroup'] = pd.cut(
           df['MonthlyCharges'],
           bins=[0, 30, 60, 90, 120],
           labels=['0-30', '31-60', '61-90', '91-120'],
           include_lowest=True
       )

[168]: print(df['ChargeGroup'].isnull().sum())
       print(df['ChargeGroup'].value_counts())

       0
       ChargeGroup
       61-90     34163
       31-60     21123
       0-30      10118
       91-120     9596
       Name: count, dtype: int64
```

Now, the current (df) has been cleaned and prepared successfully.

3) Exporting the file and verify that the file is saved and returning the size of the file.

```
:  df.to_excel(
       r"D:\EXCEL PROJECTS\Fifth New Project\python_telecom_dataset.xlsx",
       index=False
   )

:  print(os.path.getsize(
       r"D:\EXCEL PROJECTS\Fifth New Project\python_telecom_dataset.xlsx"
   ))

   7075784
```

Here, the “os” module was already imported.